Redirecting Output in Python

- Redirecting output in Python means sending the output of a program to a destination other than the screen.
- Python displays output on the screen using the print() function.
- When we redirect the output, we can send it to a file or another destination instead.
- For example, we can redirect output to:
  - A text file.
  - A log file.
  - Another output stream.

1.Normal output using print()

Example:

```
print("Hello Python")
print("Welcome to Programming")
```

Output on the screen:

```
Hello Python
Welcome to Programming
```

- Here, the output is displayed on the screen because print() sends its output to the standard output stream, called sys.stdout, by default.

2. Redirecting output to a file

- We can redirect output to a file using the file parameter of the print() function.

Example 1:

```
f = open("output.txt", "w")
print("Hello Python", file=f)
print("Welcome to Programming", file=f)
f.close()
```

- open("output.txt", "w") creates a file named output.txt or overwrites it if it already exists.
- file=f sends the printed text to the file instead of the screen.
- f.close() closes the file after writing.

- The file output.txt will contain:
  - Hello Python
  - Welcome to Programming

Example 2:

```
with open("student.txt", "w") as f:
print("Student Name: Anu", file=f)
print("Course: Python", file=f)
print("Mark: 90", file=f)
print("Output written to file successfully")
```

Output on the screen:

Output written to file successfully

3. Redirecting output using the sys module

- Python provides the sys module, which allows us to access and change the standard output stream.

Example 1:

```
import sys
f = open("output.txt", "w")
sys.stdout = f
print("Hello Students")
print("This output goes to a file")
f.close()
```

Example 2:

```
import sys

original = sys.stdout
with open("log.txt", "w") as f:
    sys.stdout = f           # redirect
    print("Line 1")          # -> log.txt
    print("Line 2")          # -> log.txt
    sys.stdout = original    # restore

print("Back on screen")
```

4.contextlib.redirect_stdout

Example:

```
from contextlib import redirect_stdout

def report():
    print("Sales: 1200")
    print("Profit: 300")

with open("report.txt", "w") as f:
    with redirect_stdout(f):
        report()        # both lines -> report.txt

print("Report saved")   # screen
```

- contextlib is a built-in Python module that provides tools for managing resources and temporary changes.
- redirect_stdout(), which temporarily changes where Python sends its standard output.
- redirect_stdout is a function available in this module.
- It allows us to redirect the output of print() from the screen to another destination, such as a file.
- When report() is called normally, both messages appear on the screen.
- However, in the given program, the function is called inside redirect_stdout(), so its output goes to a file instead.

Difference between normal output and redirected output

Normal output	Redirected output
Output appears on the screen.	Output goes to another destination.
Uses print() by default.	Can use print(..., file=f) or sys.stdout.
Useful for displaying results to users.	Useful for saving results, reports and logs.